

Formalizing Abram's worry

Martín Soto

August 14, 2023

When I am considering this concrete branch B_1 (large belief), I still have some small belief S about the structure of certain decision problems, that also applies to this one (for example, the belief "if in the corresponding counterfactual branch I Pay up, the utility in this branch will be higher"). So when I additionally condition on some action-point in that other branch $\mathbf{A}(B_2) = a$ (large belief) (for example, "in the corresponding counterfactual branch I Pay up"), I want to actually see that impact that S predicted.

It seems like the small belief will look like $S \equiv P(B_i, B_j) \rightarrow (\mathbf{A}(B_j) = a \rightarrow U(B_i))$, that is, "for any decision nodes of this structural shape, this counterfactual action has this counterfactual consequence". Of course, it might still be $\mathbb{P}$ has no idea about whether B_1 and B_2 have this shape, and so acts wrong. But that'd be not seeing the basic structure of the decision problem, and we probably want to always allow that (similarly to how our agent / inductor can consult arbitrary places of the decision tree at any time). So, since our frozen prior has only seen a finite part of the tree by the time it's frozen, and we don't want it to walk in blind, we probably want to always maximize something like $\mathbb{E}(U(B_1) | \text{"}B_1 \text{ and } B_2 \text{ look exactly like this"} \wedge \mathbf{A}(B_2) = a)$, instead of only $\mathbb{E}(U(B_1) | \mathbf{A}(B_2) = a)$. In that case, provided our prior has some belief like "all nodes that look like that have the property P ", indeed by propositional consistency $\mathbb{P}(U(B_1) = 1M\$ | \text{"}B_1 \text{ and } B_2 \text{ look exactly like this"} \wedge \mathbf{A}(B_2) = a)$ will be at least as high as our belief in S .¹

I first explain a central worry of mine.

But here we do have a problem: I don't think our prior will know "all nodes that look like that have the property P " for most nodes. Indeed, if we simply condition on "the exact explicit description of B_1 and B_2 " as above², why should our extrapolation procedure have opinions on whether this has property P ? This feels like having beliefs on a concrete very large $\underline{i}$ possessing a certain characteristic (being divisible by 3): that requires additional computation (and our prior has only performed a finite amount of computation, so it will miss out on infinitely many things). So the hope here would be that our extrapolation has magically hard-coded, for every B_i and B_j that possess that property, a sentence asserting they do, since that was decision-theoretically

¹Strictly speaking, as our belief in S conditioned on $\mathbf{A}(B_2) = a$, but allegedly S is "a general structural feature we've mathematically learned", and doesn't depend on concrete parts of the tree or concrete behavior.

²The alternative is conditioning directly on $P(B_1, B_2)$, of course, but this amounts to trusting that logical information is always presented in the most convenient way for us. What if instead of P , our inductor had written these beliefs as an equivalent but syntactically different P' ?

relevant. Similarly to the small reasoner choosing the policy "in this case do more computation". But how do we separate what is decision-theoretically relevant? Well, I can imagine introducing syntactic variables $D(n, m)$, providing a description of the m th node one can reach at the n th round (with description of any logical coins, entanglements, etc.), and similarly $U(n)$ for utility obtained in round n . But even then we cannot implement logical omniscience about D and U and their properties, since that is logical omniscience about everything. We need a syntactic measure of what is relevant.

Maybe our small reasoner instead has opinions over policies in this kind of situation, of the kind "if a decision problem has this property, then this policy on its nodes is the best one". And we check its beliefs, and if we find one such belief matching a property that the present decision problem indeed has, then our agent implements that best policy. And when several such sentences conflict we pick the one with bigger (longer) policy that still fits our situation? And if they still conflict just one at random?

I now explain what I think was your central worry.

Your additional worry was about how conditioning on that policy point affects the average probability of "me paying up". Allegedly, we only worry about this because Ω might calculate our payoff in this way.

Indeed, that's kind of what happens in our LI framework: Ω thinks specifically about that one policy point, $\mathbf{A}(B_2) = a$, but the beliefs of its inductor will also be informed by other policy points it has observed, and possibly by other mathematical properties of our algorithm (of which $\mathbf{A}$ can obtain evidence both thanks to its knowledge about algorithms in general and what this says about its source code, and thanks to conditioning on concrete action points). But ultimately, in our framework, $\mathbf{A}$ will just be trying to predict Ω 's LI. Maybe it has a lot of detailed knowledge about that LI, so that it knows this policy point in particular will affect its assessment less because of some weird quirk. Or maybe it knows next to nothing about the LI, and so falls back on using its own probability of the event, trusting that the LI is approximately as good as itself at this prediction. But ultimately all considerations about "how this affects the average" are mathematical predictions about the behavior of Ω 's LI.

So in this framework, I'm not exactly sure what "this decision is correlated to all other similar decisions" might mean, other than $\mathbf{A}$ thinking that Ω 's LI believes that.

So what if that ("all of these are correlated", or " Ω believes that all of these are correlated") is a small belief about large beliefs? Well, we'd want to, when conditioned on the large beliefs, "propagate correctly" those small beliefs. That is, if we believe it about B_i s and B_j s of that shape in general, believe it about B_1 and B_2 concretely, because they have that shape. But that leads to my implementation worry above, which is ultimately "apparently you need more computation to chart the decision-relevant equivalences / properties".